

DA COMPANION · PYTHON FOR DATA SCIENCE

Data Literacy & Analysis

Read alongside the lecture slides · Every slide example worked out in full

How to use this companion. Each section follows the same colour code. **Blue** boxes give the plain-words intuition. **Amber** boxes give a real-life analogy. **Green** boxes work through a full example, step by step. **Red** boxes warn you about common mistakes. **Purple** boxes give the one sentence to remember. Read each section in order: the analogy comes before the math, and the math comes before the worked example.

Contents

1	What is Data Analytics?	2
1.1	The four types of data analytics	2
2	Types of Data	3
2.1	Numeric data	3
2.2	Categorical data	3
2.3	Tabular data	3
2.4	Text data	3
2.5	Visual data	3
3	Exploratory Data Analysis (EDA)	5
3.1	What EDA checks for	5
3.2	A typical EDA process	5
3.3	EDA techniques: quantitative vs. graphical	6
4	Handling Categorical and Ordinal Features	7
4.1	One-hot encoding (nominal categories)	7
4.2	Label encoding (ordinal categories)	7
5	NumPy — Working with Arrays	8
5.1	Key NumPy concepts	8
6	Data Visualization — Matplotlib and Seaborn	9
6.1	Matplotlib — the foundation	9
6.2	Seaborn — statistical charts made easy	10
7	Pandas — Working with Datasets	11
7.1	Core Pandas operations	11
8	Storytelling with Data	13
8.1	How to build a data story	13
	Glossary & Symbols	14

1 What is Data Analytics?

Data analytics means studying data to find answers. You collect data, clean it, look for patterns, and then show what you found. The goal is to help people make better decisions.

The intuition

Data analytics is the art of turning raw numbers into answers. You look at a pile of data, find the patterns hiding inside, and then tell a clear story about what those patterns mean.

★ Everyday picture

Think of a doctor reading blood-test results. The numbers alone mean little. The doctor looks for patterns — high cholesterol, low iron — and uses those patterns to decide on treatment. Data analytics is the same move: read the numbers, find the story, act on it.

1.1 The four types of data analytics

There are four kinds. Each answers a different question.

Type	Question answered	Example
Descriptive	What happened?	Total sales last month were \$50 000.
Diagnostic	Why did it happen?	Sales dropped because the website was slow.
Predictive	What will happen?	Next month's sales will likely be \$48 000.
Prescriptive	What should we do?	Speed up the website to recover lost sales.

Where this shows up in ML. Machine learning uses all four types. A regression model is *predictive*. A recommendation engine is *prescriptive*. Understanding which type you need tells you which tool to reach for.

✓ Key takeaway

The four types form a ladder: describe → diagnose → predict → prescribe. Each rung needs the one below it.

2 Types of Data

Not all data looks the same. Knowing the shape of your data tells you which tools to use.

The intuition

Data comes in different shapes: numbers you can add up, categories you can count, images, text, audio, and more. Each shape needs different handling.

2.1 Numeric data

Numeric data is any value you can measure or count. Examples: salary (\$45 000), temperature (37 °C), age (22). You can add, subtract, and find averages with it.

2.2 Categorical data

Categorical data puts things into groups. Examples: department (HR, Sales, IT), colour (red, blue), yes/no. You *cannot* add categories together — "HR + Sales" is meaningless.

There are two sub-types:

- **Nominal** — no natural order. Colour, city, department.
- **Ordinal** — a clear order exists. Rating (1–5 stars), education level (school → bachelor → master).

★ Everyday picture

Think of a school grade book. A student's exam score (82/100) is numeric — you can find the class average. Their grade letter (A, B, C) is ordinal — you know A is better than B, but "A + B" means nothing. Their school house (Red, Blue, Green) is nominal — there is no order at all.

2.3 Tabular data

Tabular data is structured data organised in rows and columns. Every row is one record (one employee, one transaction). Every column is one attribute (salary, department, hire date). CSV files and spreadsheets are the most common formats.

2.4 Text data

Text data is made of words and sentences. It is *unstructured* — there are no neat rows and columns. Examples: emails, customer reviews, social media posts. We use *NLP* (Natural Language Processing) algorithms to analyse text.

2.5 Visual data

Visual data includes images and video. Images are grids of pixels; videos are sequences of image frames. Computer vision models process visual data frame by frame.

✗ Watch out

Do not mix up data types in your analysis. Applying a mean to a nominal column (e.g. averaging department codes 1, 2, 3) gives a number that means nothing. Always check what type each column is before computing statistics on it.

Where this shows up in ML. The data type decides the model. Numeric targets → regression. Categorical targets → classification. Text → NLP models (BERT, GPT). Images → convolutional neural networks (CNNs).

✓ Key takeaway

Always identify the data type of every column before you start. It decides every step that follows.

3 Exploratory Data Analysis (EDA)

You have a new dataset. Before you build any model, you need to understand what is in it. That is what EDA does.

The intuition

EDA is your first conversation with the data. You ask: are there gaps? Are there odd values? What shape does each column take? You do not yet answer the big question — you just learn what you are dealing with.

★ Everyday picture

Imagine you just moved into a new flat. Before you unpack, you walk through every room. You check for damp walls, broken sockets, and missing furniture. EDA is that walkthrough — you survey the data before you start working with it.

3.1 What EDA checks for

1. **Data distribution** — what range of values does each column take?
2. **Missing values** — which cells are empty, and how many?
3. **Outliers** — are there values that are unusually high or low?
4. **Duplicate rows** — are any records repeated?
5. **Encoding** — how are categories stored? Numbers? Text?
6. **Scale** — are the columns on very different scales (e.g. age 0–100 vs. salary 0–200 000)?

3.2 A typical EDA process

1. Load the data and check its shape (rows × columns).
2. Look at the first few rows with `df.head()`.
3. Get summary statistics with `df.describe()`.
4. Check for missing values with `df.isnull().sum()`.
5. Plot distributions (histograms, box plots).
6. Look for correlations between columns.
7. Fix or flag problems found above.

Worked example: Employee dataset — first EDA steps

Suppose you have this employee table:

EmployeeID	Department	YearsAtCompany	Salary
E001	HR	3	45 000
E002	Sales	7	62 000
E003	IT	—	58 000
E004	HR	2	46 000
E005	Sales	15	95 000

Step 1. Check shape. 5 rows, 4 columns. Columns: one ID, one nominal category, one

numeric, one numeric.

Step 2. Summary statistics for Salary. Min = \$45 000, Max = \$95 000, Mean = $\$(45+62+58+46+95)/5 = \$61\,200$.

Step 3. Spot the missing value. E003 has no `YearsAtCompany`. We need a strategy: fill with the median (4 years) or drop the row.

Step 4. Spot the potential outlier. E005 earns \$95 000 — much higher than the rest. It may be a senior employee, not an error. Flag it for review.

Step 5. Check categorical values. Departments are HR, Sales, IT. No typos, no unexpected values.

Step 6. Decision. Fill the missing year with the median (4). Keep E005 but note it is a high earner.

3.3 EDA techniques: quantitative vs. graphical

Technique type	Method	What it tells you
Quantitative	Mean, median, mode	Central value of a column
Quantitative	Standard deviation	How spread out the values are
Quantitative	Min / max / quartiles	Range and outlier boundaries
Quantitative	Correlation matrix	Which columns move together
Graphical	Histogram	Distribution shape of one column
Graphical	Box plot	Median, spread, and outliers
Graphical	Scatter plot	Relationship between two numeric columns
Graphical	Bar chart	Counts or totals across categories
Graphical	Heatmap	Correlation between many columns at once

Where this shows up in ML. EDA is not optional before building a model. A missing-value column fed to a neural network will crash training. An unscaled column will bias gradient descent. Good EDA prevents hours of debugging later.

✓ Key takeaway

EDA is not analysis — it is *preparation for* analysis. Always do it before building any model.

4 Handling Categorical and Ordinal Features

Models work with numbers, not words. So "HR", "Sales", and "IT" must be turned into numbers before training. That process is called *encoding*.

The intuition

Encoding translates categories into numbers that a model can use. The key rule: ordinal categories keep their order; nominal categories do not get a fake order.

4.1 One-hot encoding (nominal categories)

For nominal data, create one new column per category. Put a 1 in the matching column and 0 everywhere else.

Worked example: One-hot encoding the Department column

Original column has three values: HR, Sales, IT.

Step 1. List the unique values. HR, Sales, IT — three values, so three new columns.

Step 2. Create the columns. Dept_HR, Dept_Sales, Dept_IT.

Step 3. Fill the values. For E001 (HR): Dept_HR=1, Dept_Sales=0, Dept_IT=0. For E002 (Sales): Dept_HR=0, Dept_Sales=1, Dept_IT=0.

Step 4. Drop the original column. The word column is replaced by the three numeric columns.

Step 5. Result. The model can now read the department as numbers with no fake ordering.

4.2 Label encoding (ordinal categories)

For ordinal data, assign a number that respects the order.

Example: Education level → School = 1, Bachelor = 2, Master = 3, PhD = 4.

Watch out

Do *not* apply label encoding to nominal data. If you encode HR = 1, Sales = 2, IT = 3, the model will think IT is "three times more" than HR. That is wrong. Use one-hot encoding for nominal columns.

Where this shows up in ML. Decision trees can handle raw categories, but most models (linear regression, neural networks) require numeric input. Getting encoding wrong is one of the most common causes of poor model performance.

Key takeaway

Nominal → one-hot encode. Ordinal → label encode. Never swap them.

5 NumPy — Working with Arrays

Python's built-in lists are slow for math. NumPy gives us a faster, smarter array.

The intuition

A NumPy array is like a spreadsheet column, but the computer can do math on the whole column at once — without a slow loop.

★ Everyday picture

Imagine adding 10 to every price in a 10 000-row price list. With a regular Python loop you visit each cell one by one. With NumPy you write `prices + 10` and the whole column updates in one step. That is the power of *vectorisation*.

5.1 Key NumPy concepts

- **ndarray** — the core object. A grid of values, all the same type.
- **Shape** — the size in each dimension. `(3,)` is a 1-D array of 3 values. `(2, 4)` is a 2-row, 4-column matrix.
- **dtype** — the data type of the values (`int64`, `float64`, etc.).
- **Vectorised operations** — math applied to every element at once (`arr * 2`, `arr + arr`).
- **Broadcasting** — NumPy stretches a smaller array to match the shape of a larger one.

Worked example: Basic NumPy operations on a salary array

We have four salaries: 45 000, 62 000, 58 000, 95 000.

Step 1. Create the array.

```
import numpy as np
salaries = np.array([45000, 62000, 58000, 95000])
```

Step 2. Find the mean.

$$\bar{x} = \frac{45000 + 62000 + 58000 + 95000}{4} = \frac{260000}{4} = \mathbf{65000}$$

Code: `np.mean(salaries)` → **65 000.0**

Step 3. Apply a 10% raise to all salaries. `salaries * 1.10` → `[49500, 68200, 63800, 104500]`

Every element multiplied in one step — no loop needed.

Step 4. Find values above the mean. `salaries[salaries > 65000]` → `[95000]`

Only the senior employee clears the mean.

Where this shows up in ML. Every ML library (Pandas, scikit-learn, TensorFlow, PyTorch) stores data as NumPy arrays or tensors (which are NumPy-like). Knowing NumPy means you understand what is happening under the hood.

✓ Key takeaway

NumPy replaces slow Python loops with fast, one-line array operations. It is the foundation that every other data science library builds on.

6 Data Visualization — Matplotlib and Seaborn

Numbers alone are hard to interpret. A good chart shows the pattern instantly.

The intuition

Visualization turns a table of numbers into a picture. Your brain spots trends and outliers in a chart far faster than in a column of figures.

★ Everyday picture

Think of a weather forecast. A table of temperatures for every hour of the day is hard to read. A line chart shows at a glance: morning is cold, afternoon is warm, evening cools again. The same information, much easier to use.

6.1 Matplotlib — the foundation

Matplotlib is the oldest and most flexible Python plotting library. You control every detail: colours, axes, labels, sizes.

Common plot types:

Plot type	Use it when...	Matplotlib command
Line plot	Showing a trend over time	<code>plt.plot(x, y)</code>
Bar chart	Comparing categories	<code>plt.bar(categories, values)</code>
Histogram	Showing a distribution	<code>plt.hist(data, bins=20)</code>
Scatter plot	Checking two numeric columns relate	<code>plt.scatter(x, y)</code>
Box plot	Showing spread and outliers	<code>plt.boxplot(data)</code>

Worked example: Histogram of employee salaries

We want to see how salaries are spread across 20 employees.

Step 1. Import the library and create the data.

```
import matplotlib.pyplot as plt
import numpy as np
np.random.seed(42)
salaries = np.random.normal(loc=60000, scale=15000, size=20)
```

Step 2. Plot the histogram.

```
plt.figure(figsize=(7, 4))
plt.hist(salaries, bins=8, color='steelblue', edgecolor='white')
plt.xlabel('Salary ($)')
plt.ylabel('Number of employees')
plt.title('Distribution of employee salaries')
plt.tight_layout()
plt.savefig('figures/fig_salary_hist.png', dpi=150)
plt.show()
```

Step 3. Read the chart.

Most salaries cluster around \$60 000 (the mean). The bars thin out at the extremes. If one bar is isolated far from the rest, that is likely an outlier.

6.2 Seaborn — statistical charts made easy

Seaborn is built on top of Matplotlib. It makes statistical charts (box plots, violin plots, pair plots, heatmaps) with far less code.

Worked example: Box plot of salary by department

Step 1. Set up the data and import Seaborn.

```
import seaborn as sns
import pandas as pd
df = pd.DataFrame({
    'Department': ['HR', 'HR', 'Sales', 'Sales', 'IT', 'IT'],
    'Salary': [45000, 46000, 62000, 95000, 58000, 72000]
})
```

Step 2. Draw the box plot.

```
sns.boxplot(x='Department', y='Salary', data=df, palette='muted')
```

Step 3. Read the chart.

The box shows the middle 50% of salaries. The line inside is the median. Any dot outside the whiskers is an outlier. Here Sales has the widest spread and the highest outlier (\$95 000).

Where this shows up in ML. Visualization is used at every stage: during EDA to understand features, during training to watch the loss curve, and after training to evaluate results (confusion matrix heatmap, ROC curve).

Key takeaway

Use Matplotlib when you need full control. Use Seaborn when you want a clean statistical chart in two lines of code.

7 Pandas — Working with Datasets

Real data comes in files: CSVs, Excel sheets, JSON feeds. Pandas loads them into a *DataFrame* — a table you can slice, filter, and transform with one line of code.

The intuition

A Pandas *DataFrame* is a spreadsheet inside Python. Each column is a *Series* — a labelled array. You can filter rows, add columns, group by category, and merge tables — all without leaving Python.

★ Everyday picture

Think of Pandas as Excel with superpowers. You can open a million-row file, filter it to only the rows you want, and compute group averages — in about three lines of code. No clicking, no waiting for Excel to freeze.

7.1 Core Pandas operations

Operation	Code	What it does
Load a CSV	<code>pd.read_csv('file.csv')</code>	Read data into a <i>DataFrame</i>
First few rows	<code>df.head()</code>	Show the first 5 rows
Shape	<code>df.shape</code>	(rows, columns)
Column names	<code>df.columns</code>	List all column names
Summary stats	<code>df.describe()</code>	Mean, std, min, max per column
Missing values	<code>df.isnull().sum()</code>	Count of NaNs per column
Filter rows	<code>df[df['Salary'] > 60000]</code>	Keep rows where condition is true
Add a column	<code>df['Tax'] = df['Salary'] * 0.2</code>	Create a new column
Group and aggregate	<code>df.groupby('Dept')['Salary'].mean()</code>	Average salary per department
Drop missing	<code>df.dropna()</code>	Remove rows with any NaN
Fill missing	<code>df.fillna(df.median())</code>	Replace NaN with column median

Worked example: EDA on the employee DataFrame

We have a CSV with columns: EmployeeID, Department, Salary, YearsAtCompany.

Step 1. Load and inspect.

```
import pandas as pd
df = pd.read_csv('employees.csv')
print(df.shape)      # e.g. (200, 4)
print(df.head())
print(df.dtypes)
```

Step 2. Check for missing values.

```
print(df.isnull().sum())
# YearsAtCompany    12
# Everything else    0
12 employees have no YearsAtCompany value.
```

Step 3. Fill missing years with the median.

```
median_years = df['YearsAtCompany'].median() # e.g. 5.0
df['YearsAtCompany'].fillna(median_years, inplace=True)
```

Step 4. Find average salary by department.

```
dept_salary = df.groupby('Department')['Salary'].mean()
```

```
print(dept_salary)
```

Result (example):

Department

HR 47500

IT 68000

Sales 75200

IT and Sales pay more on average than HR.

Step 5. Sense-check. Are the averages in a realistic range? Is the count per group reasonable? Yes — the data looks clean after filling the missing values.

Where this shows up in ML. Pandas is the standard tool for every data step before model training. You load data with Pandas, clean it with Pandas, engineer features with Pandas, then hand a NumPy array (or a Pandas DataFrame) to scikit-learn.

✓ Key takeaway

Pandas is your Swiss Army knife for data. Load it, inspect it, clean it, reshape it — then pass clean arrays to your model.

8 Storytelling with Data

Data analysis does not end with a table of numbers. Someone needs to understand what the numbers mean and decide what to do. That is where storytelling comes in.

The intuition

A data story turns your findings into a message a non-expert can act on. You are not just reporting numbers — you are answering "so what?"

★ Everyday picture

Think of a news article about a health study. The study has thousands of rows of patient data. The journalist picks the one key finding, explains it in plain words, and shows one clear chart. That is data storytelling: one message, told simply.

8.1 How to build a data story

1. **Understand the audience.** Are they technical or not? What decision do they need to make?
2. **Collect and analyse the data.** Run your EDA, build your model, find the key result.
3. **Identify the key message.** One sentence: "Sales in the north are falling because ..."
4. **Design the visuals.** Pick the chart type that shows the message most clearly. Remove clutter.
5. **Create a narrative.** Context → finding → implication → recommendation.
6. **Refine and practise.** Can a non-expert follow it? If not, simplify further.

Worked example: Storytelling from the employee dataset

- Step 1. Finding from EDA.** Sales employees earn 58% more on average than HR employees. Sales also has the highest variance in salary.
- Step 2. Key message.** "Sales compensation is high and uneven — a few top earners pull the average up."
- Step 3. Choose the chart.** A box plot of salary by department shows both the level and the spread in one picture.
- Step 4. Build the narrative.** "We looked at 200 employee records. HR salaries cluster tightly around \$47 000. Sales salaries average \$75 000 but range from \$40 000 to \$150 000. The top 10% of Sales staff earn three times the median. Recommendation: review Sales pay bands to reduce the gap."

Where this shows up in ML. Every ML project ends with communicating results. A model with 95% accuracy is worthless if the stakeholder cannot understand what it predicts, how confident it is, and what they should do with it. Data storytelling is how you close that gap.

✓ Key takeaway

A chart without a message is decoration. Always say: "This chart shows X, which means we should do Y."

Glossary & Symbols

Key terms.

Analytics	Studying data to find patterns and support decisions.
Categorical	Data that belongs to named groups with no arithmetic meaning.
DataFrame	A Pandas table: rows are records, columns are features.
Descriptive	The type of analytics that summarises what already happened.
EDA	Exploratory Data Analysis — surveying data before modelling.
Encoding	Turning category labels into numbers a model can use.
Label encoding	Assigning ordered integers to ordinal categories.
ndarray	A NumPy multi-dimensional array.
NLP	Natural Language Processing — algorithms that work with text.
Nominal	Categorical data with no natural order (e.g. city, colour).
NumPy	Numerical Python; the fast array library at the base of data science.
One-hot	Encoding that gives each category its own 0/1 column.
Ordinal	Categorical data with a clear order (e.g. small/medium/large).
Outlier	A value far outside the usual range; may be an error or genuine.
Pandas	Python library for loading, cleaning, and reshaping tabular data.
Prescriptive	Analytics that recommends a specific action.
Predictive	Analytics that forecasts future values.
Series	A single labelled column in a Pandas DataFrame.
Vectorisation	Applying an operation to an entire array in one step (no loop).
Visualisation	Representing data as a chart or graph.

Common Pandas / NumPy calls at a glance.

Call	What it does
<code>np.array([...])</code>	Create a NumPy array from a list
<code>np.mean(arr)</code>	Arithmetic mean of an array
<code>arr[arr > x]</code>	Boolean mask: keep only elements above <code>x</code>
<code>pd.read_csv('f.csv')</code>	Load a CSV file into a DataFrame
<code>df.describe()</code>	Count, mean, std, min, quartiles, max per column
<code>df.isnull().sum()</code>	Number of missing values per column
<code>df.fillna(v)</code>	Replace every NaN with the value <code>v</code>
<code>df.groupby(col).mean()</code>	Mean of each column, grouped by <code>col</code>
<code>plt.hist(data, bins=n)</code>	Matplotlib histogram with <code>n</code> bins
<code>sns.boxplot(x=, y=, data=)</code>	Seaborn box plot split by a category